

Fig. 5: All RTOSs/RTKs have a hardware-abstraction layer (HAL) between the hardware and OS, so that porting OS becomes easy

tify whether the problem is related to hardware or software. If the problem lies with hardware, it displays the fault and calls the attention of the user for a replacement. Meanwhile, the copy that took over the system controls and runs as usual, so that the main functionality of the system does not suffer. Fig. 4 shows the sequence of events.

It is the system software's responsibility to maintain the log that the faulty unit has been replaced with a good one and the system has returned to duplex mode.

At this stage we need to understand that dual CPU implementation has two variations in their working, based on software implementation.

An FT system is fully-controlled by one CPU and the other CPU takes over when the active one fails. This is known as hot-stand-by architecture. Here, the software is simple and two critical elements need to be handled, namely, the take-over portion of the software and system/user parameters update through the interlink.

The second variation, which is complex in software implementation but more precise, is known as load-sharing architecture. In this architecture, both CPUs execute the system software, while the active one controls the system. The status of the system is almost identical and the CPU load is shared for better functionality. This architecture helps in mission-critical systems for faster take-over and control.

The system software is very critical, despite the choice of architecture. Since most FT systems are real-time systems, these use real-time operat-

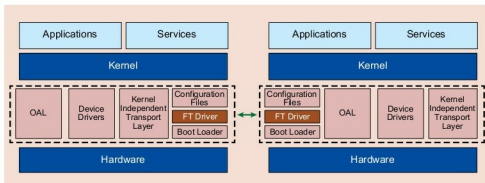


Fig. 6: High-level software architecture for FT systems

ing systems (RTOSs) or real-time kernels (RTKs). This complicates the development of FT software. Typically, all RTOSs/RTKs have a hardware-abstraction layer (HAL) (Fig. 5) between the hardware and OS, so that porting OS becomes easy. This software is written specifically for each processor, so that application developers are completely decoupled from the processors used.

When an FT system is being implemented, its performance is dependent on the way FT software is implemented in a classical RTOS/RTK application. Integrating it with HAL will give the fastest performance when handling WD timer run-out. However, updates through interlink need a complete link-handling driver.

Since the interconnect link can be anything including a simple serial port, an Ethernet link or a custom parallel bus, the update section of the software is critical. The challenge is that the software engineer, who is building the FT software, needs to work with the hardware team, as well as understand the system requirement, to implement the interlink update software.

Let us say we are implementing an FT system for a telephone exchange. From the revenue point of view, both duplicated controllers should be consistent for all billing data of on-going calls. With a metering data resolution of one second, and let us say that this controller handles about 500 customers, link speed of 2Mbps to 5Mbps will be good for consistency. The choice of interlink is based on the system's

need and the smallest resolution that the system has to handle. This is a critical aspect that every implementer should be aware of.

Often, the interlink speed and performance requirement is an afterthought, leading to sub-optimal performance of the system. Fig. 6 shows the high-level software architecture for FT systems.

For mission-critical systems, like avionics, railway-signalling controllers, medical devices and nuclear plant systems, a failure may be life-threatening. These systems have multiple CPUs (three or more) and use a complex, majority logic based voting system to implement the FT system.

CON-MON architecture. One of the unique architectures frequently used in avionics and other systems is CON-MON architecture. The name CON-MON stands for control-and-monitor processor architecture. This architecture is not fault-tolerant as the main function of this architecture is to sound an alarm when the main CPU fails. It uses two CPUs—the main CPU, which controls the function of the system, and a small microcontroller, which monitors the main CPU through the WDT run-out.

You may think what the advantage of this architecture is. Let us assume that the small microcontroller is not there. When the main CPU restarts due to a fault, details about the fault, like the time and duration, are lost, and failure is known only when the main CPU stops working. With the CON-MON architecture, the smaller controller will log this data, apart from raising the alarm. ●